

CHƯƠNG 1: LÀM VIỆC VỚI CƠ SỞ DỮ LIỆU TRONG JAVA

Mục tiêu:

Cung cấp cho sinh viên kỹ năng lập trình cơ sở dữ liệu trong java, thực hiện kết nối và tương tác các hệ quản trị cơ sở dữ liệu khác nhau, sử dụng thành thạo các JDBC API giúp các ứng dụng Java có thể kết nối và thực hiện các thao tác truy vấn và xử lý dữ liệu.

Nội dung:

- Kiến trúc JDBC
- Driver JDBC
- Kết nối và quản lý kết nối.
- Thực hiện truy vấn SQL.
- Xử lý kết quả.
- Quản lý giao dịch

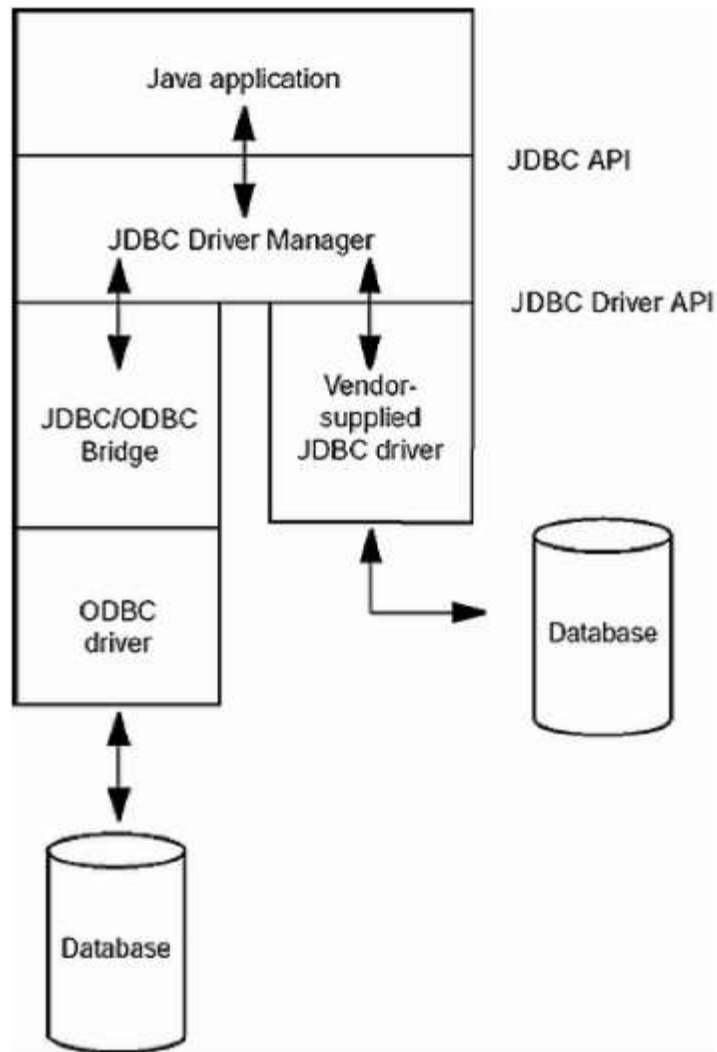
Bài 1: Kết nối cơ sở dữ liệu.

Trong bài này chúng ta sẽ tìm hiểu các vấn đề liên quan đến kiến trúc JDBC (Java Database Connectivity), Driver, và phương thức khởi tạo đối tượng Connection và các tham số để khởi tạo đối tượng đối tượng Connection, các ngoại lệ Exception khi nạp Driver và khởi tạo đối tượng Connection. Sau khi thực hiện khởi tạo Connection thành công sẽ đến bước thực hiện truy vấn đơn giản và xử lý kết quả truy vấn ở dạng đơn giản.

1.1. Kiến trúc jdbc

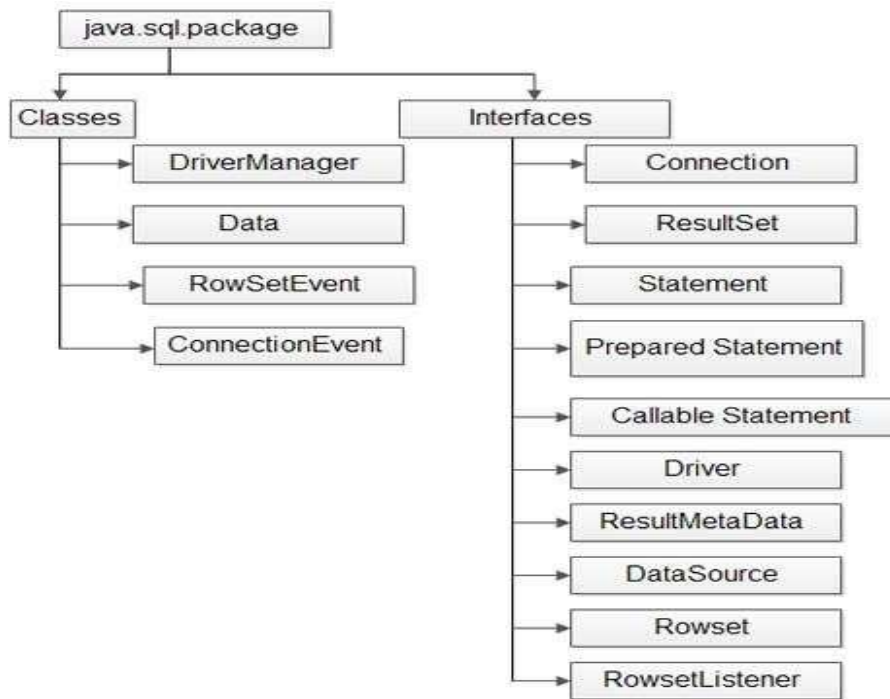
JDBC (Java Database Connectivity, java.sql.*) là một thư viện chuẩn dùng để truy cập dữ liệu quan hệ từ ứng dụng bằng ngôn ngữ truy vấn sql, một vài hệ cơ sở dữ liệu phổ biến như Access, MS SQL, Oracle, ...

Kiến trúc của JDBC gồm 2 tầng:



Hình 1: 1Kiến trúc JDBC

- + JDBC API: có nhiệm vụ chuyển các câu lệnh SQL cho bộ quản lý trình điều khiển JDBC
- + JDBC Driver API: thực hiện nhiệm vụ liên hệ với trình điều khiển của hệ quản trị cơ sở dữ liệu cụ thể.
 - Các lớp và giao diện cơ bản trong JDBC API.



Hình 1: 2 Các lớp và giao diện cơ bản trong JDBC

1.2. Các trình điều khiển (jdbc driver)

Trình điều khiển JDBC là đoạn chương trình, do chính cầu hệ quản trị CSDL cụ thể thực hiện các câu lệnh SQL. nhà xây dựng hệ quản trị CSDL hoặc do nhà cung ứng thứ ba cung cấp, có khả năng yêu

Danh sách các trình điều khiển JDBC cho các hệ quản trị CSDL khác nhau được Sun cung cấp và cập nhật liên tục tại địa chỉ:

<http://industry.java.sun.com/products/jdbc/drivers.>

Các trình điều khiển JDBC được phân làm 04 loại khác nhau.

-Loại 1: **Bridge Driver** kết nối với các hệ CSDL thông qua cầu nối ODBC, dùng trong ngày đầu Java xuất hiện, ngày nay không còn phổ biến do có nhiều hạn chế. Nó cung cấp kèm trong bộ J2SE với tên: `sun.jdbc.odbc.JdbcOdbcDriver`.

-Loại 2: **Native API Driver**: chuyển các lời gọi của JDBC API sang thư viện hàm (API) tương ứng với từng hệ CSDL cụ thể. Trình điều khiển do nhà xây dựng hệ CSDL cung cấp.

-Loại 3: **JDBC-Net Driver**: chuyển các lời gọi JDBC API sang một dạng chuẩn độc lập với các hệ CSDL, và sau được chuyển sang lời gọi của hệ CSDL cụ thể bởi 1 chương trình trung gian. Trình điều khiển của các nhà cung ứng thứ 3 thường thuộc loại này. Lợi thế của trình điều khiển loại này là không cần cung cấp mã lệnh kèm theo và có thể sử dụng cùng một trình điều khiển để truy cập đến nhiều hệ CSDL khác nhau.

-Loại 4: **Native Protocol Driver**: chuyển các lời gọi JDBC API sang mã lệnh của hệ CSDL cụ thể. Đây là các trình điều khiển thuần Java, có nghĩa là không cần phải có mã lệnh của hệ CSDL cụ thể khi thi hành chương trình.

1.3. Viết chương trình làm việc với csdl qua jdbc

Với 1 chương trình viết bởi ngôn ngữ Java có sử dụng JDBC cho việc giao tiếp với cơ sở dữ liệu thì bạn phải thực hiện 5 bước như sau:

- 1) Nạp trình điều khiển kết nối (Load Driver)
- 2) Thiết lập kết nối đến cơ sở dữ liệu thông qua “Connection String” (Establish a database connection)
- 3) Tạo và thực thi 1 câu lệnh truy vấn đến cơ sở dữ liệu (Create and execute an SQL Statement)
- 4) Xử lý kết quả nhận về sau khi thi hành lệnh truy vấn thành công trên cơ sở dữ liệu (Process the results)
- 5) Đóng kết nối và giải phóng tài nguyên đã dành cho kết nối đó (Close the database connection)

Bước 1: Nạp trình điều khiển kết nối (Load Driver):

Để tiến hành nạp trình điều khiển kết nối, bạn phải dùng phương thức `forName` của lớp `Class` để tạo ra 1 thể hiện của lớp kết nối dựa trên giao thức mà JDBC cung cấp. Cú pháp của lệnh này như sau

```
public static Class forName(String className) throws ClassNotFoundException
```

Ví dụ : sử dụng kết nối theo kiểu 1 của JDBC (ODBC-JDBC Bridge) thì câu lệnh để load driver cho kiểu này là:

```
Class.forName("jdbc:odbc:JdbcOdbcDriver");
```

Chú ý: Chuỗi “`jdbc:odbc:JdbcOdbcDriver`” dùng làm tham số của hàm chính là giao thức điều khiển dành cho kết nối loại 1 của JDBC.

Trình điều khiển của MySQL:

```
Class.forName("org.gjt.mm.mysql.Driver");
```

Trình điều khiển của Oracle:

```
Class.forName("oracle.jdbc.driver.OracleDriver");
```

Trình điều khiển của Sybase: `Class.forName("com.sybase.jdbc.SybDriver");`

Trình điều khiển qua cầu nối ODBC:

```
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
```

Chú ý: Nếu chúng ta có sử dụng tới ODBC, thì đầu tiên phải tạo nguồn ODBC cho ứng dụng.

Ví dụ trên windows (Access database), *Start -> Control Panel -> Administrative Tools -> Data Sources (ODBC)*

Đối với MySQL, ta phải tải một gói driver có tên MySQL Connector J JDBC driver về từ: <http://dev.mysql.com/downloads/> . Giải nén sau đó add tập tin .jar vào project, ví dụ trên Eclipse như sau:

- Trong trường hợp khác, bạn muốn kết nối đến cơ sở dữ liệu bằng kiểu kết nối thứ 4 của JDBC đối với cơ sở dữ liệu SQL Server, ta có thể thực hiện như sau

```
Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
```

Chú ý : Trong tình huống này thì chuỗi

"com.microsoft.sqlserver.jdbc.SQLServerDriver" dùng làm tham số của

hàm chính là giao thức điều khiển dành cho kết nối loại 4 của JDBC đối với cơ sở dữ liệu SQL Server

Bước 2: Thiết lập kết nối đến cơ sở dữ liệu (Establish a database connection)

Việc kết nối với CSDL được thực hiện với một trong các phương thức sau trong lớp DriverManager:

-*public static Connection getConnection(String url) throws SQLException:* thực hiện kết nối với CSDL được yêu cầu. Bộ quản lý trình điều khiển sẽ tự động lựa chọn trình điều khiển phù hợp trong số các trình điều khiển đã được nạp.

-*public static Connection getConnection(String url, String user, String pass) throws SQLException:* tiến hành kết nối tới CSDL với tài khoản user và mật mã pass.

-*public static Connection getConnection(String url, Properties info) throws SQLException:* tương tự hai phương thức trên ngoài ra cung cấp thêm các thông tin qui định thuộc tính kết nối thông qua đối tượng của lớp **Properties**.

Kết quả trả về của các phương thức trên là đối tượng của lớp java.sql.Connection được dùng để đại diện cho kết nối đến CSDL.

Ví dụ: kết nối đến Database có tên qlCuocHen đã tạo ra trong SQL Server thông qua tài khoản sa với mật khẩu là 123456 thông qua kiểu kết nối thứ nhất của JDBC. Câu lệnh có thể viết như sau

```
String strConnect = "jdbc:odbc:driver={SQL Server};Server=BODUA-D778EAB2E;  
Database =qlCuocHen; UserName=sa; Password=123456";  
Connection cnObj = DriverManager.getConnection(strConnect);
```

Chú ý: Cú pháp của chuỗi kết nối được mô tả như sau

```
"jdbc:odbc:driver={SQLServer};Server=<ServerName>; Database=<YourDB>;UserN
ame=<dbUser>;Password=<***>"
```

Tương tự, nếu sử dụng kiểu kết nối thứ nhất, bạn có thể sử dụng connection String theo cú pháp như sau

```
"dbc:sqlserver://<server_name>:<port>;databaseName=<YourDB>;user=<userName
>; password=<pass>"
```

Bước 3: Tạo và thực thi 1 câu truy vấn dữ liệu dạng SQL (Create and Execute an SQL Statement)

Khi 1 kết nối đã được thiết lập đến Database thành công, để có thể tác động lên cơ sở dữ liệu thông qua connection đó thì bạn phải tạo ra 1 đối tượng để có thể thực thi các tác động lên cơ sở dữ liệu của chương trình thông qua các câu lệnh dạng SQL, đối tượng đó có thể là object của 1 trong 3 lớp sau :

- **Statement**
- **PreparedStatement**
- **CallableStatement**

(Trong phạm vi này, tôi mô tả cho việc sử dụng đối tượng Statement)

Như vậy, sau khi thiết lập kết nối thành công, ta phải tạo ra 1 đối tượng

Statement để nhận về 1 thể hiện của lớp này thông qua phương thức *createStatement()* của đối tượng **Connection** đã thiết lập.

Cú pháp

```
public Statement createStatement() throws SQLException
Connection ketNoi = null;
Class.forName("sun.jdbc.odbc.JdbcOdbcDriver");
String Url="jdbc:odbc:driver={SQLServer};Server=BODUA-D778EAB2E; Databas
e =qlCuocHen;UserName=sa; Password=123456";";
ketNoi= DriverManager.getConnection(Url);
ResultSet kq=null;
Statement stCmd = this.ketNoi.createStatement();
```

Bước 4: Xử lý kết quả nhận về sau khi thi hành lệnh truy vấn thành công trên cơ sở dữ liệu (Process the results)

Một đối tượng của lớp **Statement**, sau khi tạo ra có thể thực thi 1 lệnh SQL tác động lên cơ sở dữ liệu thông qua phương thức *executeQuery()* (hay *executeUpdate()* – thường dùng khi cần thực hiện các câu lệnh SQL dạng DDL: Data Definition Language). Kết quả trả về khi thực thi lệnh *executeQuery* sẽ là 1 đối tượng của lớp

ResultSet chứa thông tin là kết quả truy vấn được từ cơ sở dữ liệu. Cú pháp của lệnh này như sau:

public ResultSet executeQuery(String strQuery) throws SQLException

Với 1 đối tượng **ResultSet**, bạn có thể sử dụng các phương thức tiêu biểu để thao tác đối với dữ liệu, hay điều khiển đối tượng này như sau

.next() –chuyển con trỏ sang mẫu tin kết tiếp để truy xuất dữ liệu

.getString(tenCot) – đọc thông tin của 1 trường thông tin dạng chuỗi có trong kết quả trả về của ResultSet. Tương tự đối với các kết quả dạng khác của các kiểu thuộc dạng Primitive data type như : int, boolean, byte, long, float, double ... Các phương thức tương ứng có thể lần lượt là : *.getInt(tenCot)*; *.getBoolean(tenCot)*; *.getByte(tenCot)*; *.getLong(tenCot)*; *.getFloat(tenCot)*; *.getDouble(tenCot)*... Các hàm này thường overload 2 dạng, dạng thứ nhất sử dụng tham số kiểu chuỗi chỉ ra tên của cột chứa thông tin muốn đọc, dạng thứ 2 kiểu int mô tả cho chỉ số của cột muốn đọc.

Ví Dụ:Giả sử sau khi đã tạo kết nối thành công, ta tạo 1 đối tượng của lớp Statement và đọc dữ liệu của bảng chứa thông tin là tên của các thành viên có trong **dsNguoiQuen** của database **qlCuocHen**, với các thông tin cần đọc về bao gồm các trường : **hoNQ**, **tenNQ**, **gioiTinh**, **tuoiNQ**. Ta sẽ làm như sau

```
Statement stCmd = ketNoi.createStatement();
String strQuery= "Select hoNQ, tenNQ, gioiTinh, tuoiNQ from dsNguoiQuen;";
ResultSet kq = stCmd.executeQuery(strQuery);
//-- Lặp trong khi còn đọc được dữ liệu trong ResultSet
While (kq.next()){
System.out.print(" Họ và tên :"+kq.getString("hoNQ") + " " +kq.getString("tenNQ");
System.out.print("Giới tính :"+(kq.getBoolean("gioiTinh") ?"Nam":"Nữ"));
System.out.println("Hiện nay : "+kq.getInt("tuoiNQ").toString()+" Tuổi");
}
Hoặc ta có thể truy xuất đến chỉ số của cột như sau
//-- Lặp trong khi còn đọc được dữ liệu trong ResultSet
While (kq.next()){
System.out.print(" Họ và tên :"+kq.getString(1)+" " +kq.getString(2);
System.out.print("Giới tính :"+(kq.getBoolean(3)?"Nam":"Nữ"));
System.out.println("Hiện nay : "+kq.getInt(4).toString()+" Tuổi");
}
```

Bước 5: Đóng kết nối và giải phóng tài nguyên (Close the database connection)

```
public void close() throws SQLException
```

1.4. Kiểu dữ liệu sql và kiểu dữ liệu java

Trong quá trình thao tác với CSDL, chúng ta sẽ gặp phải vấn đề chuyển đổi giữa kiểu dữ liệu trong CSDL sang kiểu dữ liệu Java hỗ trợ và ngược lại. Việc chuyển đổi này được thực hiện như trong 2 bảng sau.

SQL Type	Java Type
BIT	Boolean
TINYINT	Byte
SMALLINT	Short
INTEGER	Int
BIGINT	Long
REAL	Float
FLOAT	Double
DOUBLE	Double
DECIMAL	java.math.BigDecimal
NUMERIC	java.math.BigDecimal
CHAR	java.lang.String
VARCHAR	java.lang.String
LONGVARCHAR	java.lang.String
DATE	java.sql.Date
TIME	java.sql.Time
TIMESTAMP	java.sql.Timestamp
BINARY	byte[]
VARBINARY	byte[]
LONGVARBINARY	byte[]
BLOB	java.sql.Blob
CLOB	Java.sql.Clob
ARRAY	Java.sql.Array
REF	Java.sql.Ref
STRUCT	Java.sql.Struct

Bảng 1: 1 Bảng chuyển đổi từ kiểu dữ liệu SQL sang Java

Java Type	SQL Type
boolean	BIT
byte	TINYINT
short	SMALLINT
int	INTEGER
long	BIGINT
float	REAL
double	DOUBLE
java.math.BigDecimal	NUMERIC
java.lang.String	VARCHAR or LONGVARCHAR
byte[]	VARBINARY or LONGVARBINARY
java.sql.Date	DATE
java.sql.Time	TIME
java.sql.Timestamp	TIMESTAMP
java.sql.Blob	BLOB
java.sql.Clob	CLOB
java.sql.Array	ARRAY
java.sql.Ref	REF

Bảng 1: 2 Bảng chuyển đổi từ kiểu dữ liệu Java sang SQL

1.5. Statement và ResultSet, PreparedStatement

Sau đây chúng ta sẽ tìm hiểu kỹ hơn về hai đối tượng **Statement** và **ResultSet**. **Statement** là một đối tượng giúp thực thi những câu lệnh sql và trả về kết quả. Để có đối tượng này, chúng ta dùng hàm **createStatement** của lớp **Connection**. Một số hàm quan trọng trong **Statement** như sau:

1. **executeQuery()** thực thi một câu lệnh có lấy giá trị trả về
2. **executeUpdate()** thực thi một câu lệnh mà không lấy giá trị trả về từ database
3. **getConnection()** trả về đối tượng Connection mà đã tạo ra Statement này
4. **close()** đóng statement

ResultSet là tập các kết quả trả về khi thực thi một câu lệnh sql. Một vài phương thức quan trọng trong **ResultSet** như:

- **getXXX** (XXX là các kiểu dữ liệu cơ bản như Int, Float, String, Object, ...) dùng để lấy về dữ liệu
- **updateXXX** dùng để cập nhật dữ liệu cho **ResultSet**

- *next()* di chuyển con trỏ tới bản ghi kế tiếp từ vị trí hiện hành
- *previous()* di chuyển con trỏ ngược lại một dòng từ vị trí hiện hành
- *last()* di chuyển con trỏ tới bản ghi cuối cùng của ResultSet
- *first()* di chuyển con trỏ tới bản ghi đầu tiên trong ResultSet
- *getArray()* trả về giá trị trong một cột thành một mảng
- *insertRow()* thêm một bản ghi vào trong ResultSet và trong database
- *deleteRow()* xóa một bản ghi trong ResultSet và trong database

Hãy xem xét ví dụ sau:

```
import java.sql.Connection;
import java.sql.Driver;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
public class ConnectDemo0 {
    public static void main(String[] args) {
        Connection con = null;
        try {
            Class.forName("com.mysql.jdbc.Driver");
            con = DriverManager.getConnection("jdbc:mysql://localhost:3306
/k7g"
                , "root", "admin");
            System.out.println("connection ok");
        }
        catch (ClassNotFoundException e) {
            e.printStackTrace();
        }
        catch (SQLException e1) {
            e1.printStackTrace();
        }
        try {
            Statement stm = con.createStatement();
            String sql1 = "select * from sinhvien";
            ResultSet rs = stm.executeQuery(sql1);
            while(rs.next()){
```

```

        System.out.print(rs.getInt(1)+"-");
        System.out.print(rs.getString("hoten"+"-");
        System.out.print(rs.getString("diachi"+"-");
        System.out.print(rs.getString(4)+"-");
        System.out.print(rs.getInt(5));
        System.out.println("");
    }
    rs.close();
    stm.close();
    con.close();
}
catch (SQLException e) {
    e.printStackTrace();
}
}
}

```

PreparedStatement

PreparedStatement Interface là một interface con của **Statement**. Nó được sử dụng để thực thi các truy vấn được tham số hóa. Tất cả tham số trong JDBC được biểu diễn bởi ký tự ? (còn được biết với tên gọi là Parameter Marker). Ta phải cung cấp các giá trị cho mỗi tham số trước khi thực thi lệnh SQL.

Phương thức **setXXX()** gắn kết các giá trị tới các tham số, trong đó XXX biểu diễn kiểu dữ liệu trong Java của giá trị mà ta muốn gắn kết cho tham số đầu vào. Nếu quên cung cấp các giá trị, thì sẽ gặp **SQLException**.

Các phương thức của **PreparedStatement Interface**

Dưới đây là một số phương thức quan trọng của **PreparedStatement Interface**:

1. *public void setInt(int paramIndex, int giaTri)*

Thiết lập tham số đã cho tới giá trị nguyên trong Java đã cung cấp. Driver sẽ chuyển đổi giá trị này thành một giá trị nguyên trong SQL khi nó gửi giá trị tới Database.

Trong đó, *paramIndex* là tham số đầu tiên, tham số thứ hai, ...; *giaTri* là giá trị của tham số.

2. *public void setString(int paramIndex, String giaTri)*

Thiết lập tham số đã cho thành giá trị **String** trong Java đã cung cấp. Driver chuyển đổi giá trị này thành một kiểu VARCHAR hoặc LONGVARCHAR (tùy thuộc vào kích cỡ tham số) khi nó gửi giá trị tới Database. (Miêu tả về tham số như trên.)

3. *public void setFloat(int paramIndex, float giaTri)*

Thiết lập tham số đã cho thành giá trị float trong Java đã cung cấp. Driver chuyển đổi giá trị này thành một giá trị REAL trong SQL khi nó gửi giá trị tới Database. (Miêu tả về tham số như trên.)

4. *public void setDouble(int paramIndex, double giaTri)*

Thiết lập tham số đã cho thành giá trị double trong Java đã cung cấp. Driver chuyển đổi giá trị này thành một giá trị DOUBLE trong SQL khi nó gửi giá trị tới Database. (Miêu tả về tham số như trên.)

5. *public int executeUpdate()*

Thực thi truy vấn SQL trong đối tượng PreparedStatement này, mà phải là một lệnh DML (lệnh thao tác dữ liệu) trong SQL như INSERT, UPDATE hoặc DELETE, hoặc một lệnh SQL mà không trả về bất cứ cái gì, chẳng hạn như một lệnh DDL (như CREATE, ALTER, ...)

6. *public ResultSet executeQuery() throws SQLException*

Thực thi truy vấn SQL trong đối tượng **PreparedStatement** này và trả về đối tượng **ResultSet** được tạo bởi truy vấn. Phương thức này ném ngoại lệ **SQLException** và **SQLTimeoutException**.

```
//Buoc 1: can inport cac package can thiet
import java.sql.*;
public class ViDuJDBC {
    // Ten cua driver va dia chi URL cua co so du lieu
    static final String JDBC_DRIVER = "com.mysql.jdbc.Driver";
    static final String DB_URL = "jdbc:mysql://localhost/sinhvien";
    // Ten nguoi dung va mat khau cua co so du lieu
    static final String USER = "root";
    static final String PASS = "123456";
    public static void main(String[] args) {
        Connection conn = null;
        PreparedStatement stmt = null;
        try{
            // Buoc 2: Dang ky Driver
            Class.forName("com.mysql.jdbc.Driver");
            // Buoc 3: Mo mot ket noi
            System.out.println("Dang ket noi toi co so du lieu ...");
            conn = DriverManager.getConnection(DB_URL,USER,PASS);
            // Buoc 4: Thuc thi truy van
```

```

System.out.println("Tao cac lenh truy van SQL ...");
String sql = "UPDATE sinhvienk60 SET diemthi=? WHERE mssv=?";
stmt = conn.prepareStatement(sql);
// Gan ket cac gia tri vao trong cac tham so.
stmt.setInt(1, 6); // Lenh nay se thiet lap diemthi
stmt.setInt(2, 1); // Lenh nay se thiet lap mssv
// Bay gio cap nhat diem thi cua ban ghi co mssv=1;
int rows = stmt.executeUpdate();
System.out.println("So hang bi tac dong : " + rows );
// Lua chon tat ca ban ghi va hien thi chung.
sql = "SELECT mssv, ho, ten, diemthi FROM sinhvienk60";
ResultSet rs = stmt.executeQuery(sql);
// Buoc 5: Lay du lieu tu Result Set
while(rs.next()){
    // Lay du lieu boi su dung ten cot
    int mssv = rs.getInt("mssv");
    int diemthi = rs.getInt("diemthi");
    String ho = rs.getString("ho");
    String ten = rs.getString("ten");
    // Hien thi cac gia tri
    System.out.print("\nMSSV: " + mssv);
    System.out.print("\nHo: " + ho);
    System.out.println("\nTen: " + ten);
    System.out.print("\nDiem Thi: " + diemthi);
    System.out.print("\n=====");
}
// Buoc 6: Don sach moi truong va giai phong resource
rs.close();
stmt.close();
conn.close();
}catch(SQLException se){
    // Xu ly cac loi cho JDBC
    se.printStackTrace();
}catch(Exception e){
    // Xu ly cac loi cho Class.forName

```

```

        e.printStackTrace();
    }finally{
        // Khoi finally duoc su dung de dong cac resource
        try{
            if(stmt!=null)
                stmt.close();
        }catch(SQLException se2){
        }
        try{
            if(conn!=null)
                conn.close();
        }catch(SQLException se){
            se.printStackTrace();
        }// Ket thuc khoi finally
    }// Ket thuc khoi try
    System.out.println("\nVietJack chuc cac ban hoc tot!");
}// Ket thuc main
}// Ket thuc ViDuJDBC

```

Câu hỏi ôn tập

1. JDBC là gì và chức năng chính của nó là gì?. Mô tả JDBC và giải thích tại sao nó quan trọng trong phát triển ứng dụng Java.
2. Các bước cơ bản để thiết lập kết nối JDBC với cơ sở dữ liệu?. Liệt kê các bước cần thực hiện để thiết lập kết nối JDBC với một cơ sở dữ liệu.
3. Khác biệt giữa Statement, PreparedStatement và CallableStatement trong JDBC là gì?. Đặc điểm chính của các loại Statement trong JDBC và sự khác biệt giữa chúng.
4. Làm thế nào để thực hiện truy vấn SQL trong JDBC?. Mô tả quy trình để thực hiện một truy vấn SQL bằng JDBC và giải thích ví dụ cụ thể.
5. Tại sao sử dụng PreparedStatement là tốt hơn so với Statement trong một số trường hợp?.
6. Nêu rõ lợi ích của việc sử dụng PreparedStatement và tại sao nó được ưa chuộng hơn trong một số trường hợp.
7. Giải thích ResultSet trong JDBC và làm thế nào để xử lý dữ liệu từ ResultSet?. Định nghĩa ResultSet và mô tả cách xử lý dữ liệu trong JDBC.

Bài 2: Lập trình kết nối cơ sở dữ liệu(Tiếp).

Tiếp nối phần lập trình kết nối cơ sở dữ liệu trong phần này sẽ tiếp tục tìm hiểu một khái niệm quan trọng liên quan đến đó là Transaction (Giao dịch) ý nghĩa của giao dịch, và tìm hiểu các phương thức và thuộc tính quan trọng khác liên quan. Ngoài ra trong bài này cũng trình bày về các Exception trong lập trình kết nối cơ sở dữ liệu và cách giải quyết các ngoại lệ này.

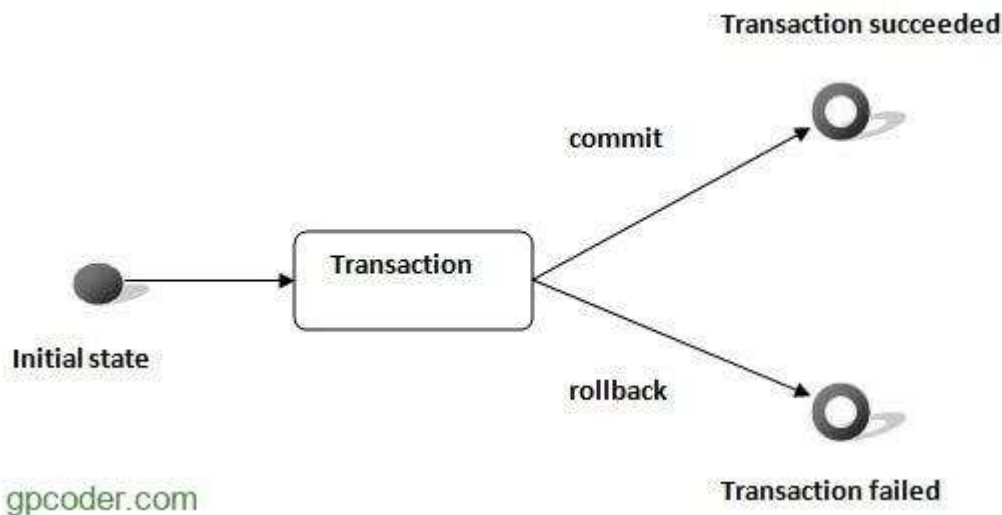
1.6. Transaction trong JDBC

Giao dịch (Transaction) là một khái niệm quan trọng trong SQL. Transaction đảm bảo tính toàn vẹn dữ liệu. Các câu lệnh trong Transaction được gọi là thành công nếu cả câu lệnh thành công. Ngược lại chỉ cần 1 trong các câu lệnh lỗi thì coi như giao dịch không thành công, phải rollback lại trạng thái ban đầu.

Ví dụ: Người A chuyển một khoản tiền 1000\$ vào tài khoản người B như vậy trong Database diễn ra 2 quá trình:

- Trừ số dư tài khoản của người A đi 1000\$.
- Thêm vào số dư tài khoản của người B 1000\$.

Nếu không có transaction thì một trong 2 quá trình lỗi gây tổn thất cho phía ngân hàng hoặc cho người dùng.



Hình 1: 3 Các trạng thái của Transaction

Các thuộc tính ACID miêu tả rõ ràng nhất về Transaction. 4 thuộc tính này bao gồm Atomicity, Consistency, Isolation và Durability, trong đó:

- Atomicity**: nghĩa là tất cả thành công hoặc không.
- Consistency**: bảo đảm rằng tính đồng nhất của dữ liệu.
- Isolation**: bảo đảm rằng Transaction này là độc lập với Transaction khác.

-Durability: nghĩa là khi một Transaction đã được ký thác thì nó sẽ vẫn tồn tại như thế cho dù xảy ra các lỗi, ...

Theo mặc định, kết nối JDBC là trong chế độ auto-commit, tức là mỗi lệnh SQL được ký thác tới Database mỗi khi nó kết thúc. Điều này là tốt với các ứng dụng nhỏ khi mà số lượng đơn vị công việc là nhỏ và không phức tạp. Với các ứng dụng trung bình và lớn, việc quản lý riêng Transaction giúp bạn cải thiện rõ rệt hiệu suất và duy trì tính toàn vẹn của tiến trình.

Trong JDBC, Connection Interface cung cấp các phương thức sau để quản lý transaction:

1. **void setAutoCommit(boolean status):** Là true theo mặc định. Để thao tác với Transaction, bạn nên thiết lập về false.
2. **void commit():** Để ký thác các thay đổi bạn đã thực hiện.
3. **void rollback():** Xóa tất cả các thay đổi đã được thực hiện trước đó và quay về trạng thái trước khi thực hiện thay đổi.
4. **setSavepoint(String ten_cua_savepoint):** Định nghĩa một savepoint mới. Phương thức này cũng trả về một đối tượng Savepoint.
5. **releaseSavepoint(Savepoint ten_cua_savepoint):** Xóa một savepoint. Phương thức này nhận một đối tượng Savepoint làm tham số.
6. **rollback (String ten_cua_savepoint):** Quay về trạng thái của savepoint đã cho.

Ví Dụ: Hãy xem xét ví dụ sau khi thực hiện insert 2 user vào database. User 1 insert thành công, User không insert được do bị lỗi SQL. Nếu không có transaction thì User 1 được lưu vào database. Chúng ta không mong muốn điều này.

```
import java.sql.Connection;
import java.sql.SQLException;
import java.sql.Statement;
public class TransactionExample {
    public static void main(String[] args) throws SQLException {
        try (Connection con = ConnectionUtils.openConnection();
            Statement st = con.createStatement();) {
            con.setAutoCommit(false); // 1. Disable individual transaction
            try {
                // This user will be inserted
                st.executeUpdate("INSERT INTO user(username, password, createdDate) "+ "VALUE('user-1', '123', now());");
            } catch (SQLException e) {
                // Rollback the transaction
                con.rollback();
            }
        }
    }
}
```

```

        System.out.println("Inserted user-1 successfully");
// This is an error sql. Cannot insert user
        st.executeUpdate("INSERT INTO user2(username, password, cr
eatedDATE) + "VALUE('user-2', '123', now());");
        System.out.println("Inserted user-2 successfully");
        con.commit(); // 2. commit data to database if all command
are success
    } catch (Exception e) {
        e.printStackTrace();
        con.rollback(); // 2. roll-back data if one of command are
failed
        System.out.println("Rollback data");
    }
}
}
}
}

```

Với ví dụ trên ta sẽ có kết quả như sau:

```

Inserted user-1 successfully
java.sql.SQLException: Table 'jdbcdemo.user2' doesn't exist
at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:120)
at com.mysql.cj.jdbc.exceptions.SQLException.createSQLException(SQLException.java:97)
at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExcepti
onsMapping.java:122)
at com.mysql.cj.jdbc.StatementImpl.executeUpdateInternal(StatementImpl.java:1335)
at com.mysql.cj.jdbc.StatementImpl.executeLargeUpdate(StatementImpl.java:2108)
at com.mysql.cj.jdbc.StatementImpl.executeUpdate(StatementImpl.java:1245)
at com.gpcoder.TransactionExample.main(TransactionExample.java:21)
Rollback data

```

Batch Processing

Batch Processing là nhóm các lệnh có liên quan vào trong một batch và thực thi chúng. Trong đó, các thao tác lệnh của chương trình được thực thi liên tiếp nhau mà không cần sự can thiệp của người dùng.

Việc ứng dụng Batch Processing trong cơ sở dữ liệu là rất tiện lợi. Khi bạn gửi một số lệnh SQL cùng một lúc, bạn đã giảm được chi phí thời gian giao tiếp và vì thế nâng cao được hiệu suất.

Một số phương thức hỗ trợ Batch Processing:

1. **addBatch(String sql)** : thêm các lệnh đơn tới batch. Tham số sql ở đây là một lệnh insert, update hoặc delete. Phương thức này không được gọi trên một đối tượng PreparedStatement và CallableStatement.
2. **int[] executeBatch()** :bắt đầu sự thực thi của tất cả các lệnh đã được nhóm lại với nhau này. Các phần tử trong mảng được trả về bởi phương thức này có thể là:

Một giá trị lớn hơn hoặc bằng 0 chỉ rằng lệnh được thực thi thành công. Và giá trị đó (gọi là update count) là số hàng trong cơ sở dữ liệu bị tác động bởi sự thực thi của lệnh.

Một giá trị SUCCESS_NO_INFO chỉ rằng lệnh được thực thi thành công nhưng không biết số hàng bị tác động bởi lệnh.

Một giá trị EXECUTE_FAILED chỉ rằng lệnh đã thực thi thất bại.

3. **clearBatch()** : xóa lệnh đã thêm khỏi batch.

Ví dụ: Một trong những trường hợp thường sử dụng tính năng này là import dữ liệu. Giả sử chúng ta có một file csv chứa danh sách user, sau khi đọc file chúng ta sẽ insert danh sách này vào database. Chúng ta có thể call executeUpdate() cho từng user. Tuy nhiên, nó sẽ gặp vấn đề về performance nếu danh sách user của chúng ta rất lớn. Một trong những cách rất hay và thường được sử dụng là Batch Processing.

```
public class BatchProcessingExample {
    public static void main(String[] args) throws SQLException {
        String sqlInsert = "INSERT INTO user(username, password, createdDate) "+ " VALUE(?, ?, ?)";
        try (
            Connection con = ConnectionUtils.openConnection();
            PreparedStatement pstmt =con.prepareStatement(sqlInsert, Statement.RETURN_GENERATED_KEYS);) {
            try {
                con.setAutoCommit(false);
                List users = readUsersFromCsvFile();
                for (User user : users) {
                    pstmt.setString(1, user.getUsername());
                    pstmt.setString(2, user.getPassword());
                    pstmt.setDate(3, user.getCreatedDate());
                    pstmt.addBatch(); // Add user to batch
                }
            }
        }
    }
}
```

```
// Executes the SQL statement
    int[] counts = pstmt.executeBatch();
    System.out.println("Affected row [0] = " + counts[0]);
    System.out.println("Affected row [1] = " + counts[1]);
    con.commit();
} catch (Exception e) {
    e.printStackTrace();
    con.rollback();
}
}
}

private static List readUsersFromCsvFile() {
    List users = new ArrayList<>();
    for (int i = 1; i <= 2; i++) {
        users.add(new User("user-" + i, "pwd123", new java.sql.Date(System.currentTimeMillis())));
    }
    return users;
}
}
```

Thực thi chương trình trên, chúng ta có kết quả sau:

```
Affected row [0] = 1
Affected row [1] = 1
```

Kiểm tra trong bảng user, chúng ta thấy 2 user đã được lưu.

1.7. Ngoại lệ SQLException

Trong JDBC, lớp java.sql.SQLException cung cấp rất nhiều phương thức để xử lý các ngoại lệ xảy ra cho cả Driver và Database. Đây là một trong các lớp cơ bản của JDBC, và chịu trách nhiệm về xử lý các ngoại lệ.

Hãy xem xét ví dụ sau:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;

public class SQLExceptionExample {
    private static final String hostName = "localhost";
```

```

private static final String dbName = "jdbcdemo";
private static final String userName = "root";
private static final String password = "";
private static final String connectionURL = "jdbc:mysql://" + hostName
+ ":3306/" + dbName;

public static void main(String[] args) {
    try {
        Class.forName("com.mysql.jdbc.Driver");
        Connection con = DriverManager.getConnection(connectionURL, us
erName, password);

        Statement st = con.createStatement();
        String sqlInsert = "INSERT INTO user(username, password, creat
edDate) "+ " VALUE('gpcoder', '123', now());";
        int numberRowsAffected = st.executeUpdate(sqlInsert);
        if (numberRowsAffected == 0) {
            System.out.println("insertion failed");
        } else {
            System.out.println("inserted successfully : " + numberRows
Affected);
        }
    } catch (SQLException se) {
        se.printStackTrace();
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

Với ví dụ trên ta có thể gặp một số ngoại lệ sau:

-Nếu Driver không tồn tại hoặc không phù hợp, ta sẽ nhận được lỗi như sau:

```

java.lang.ClassNotFoundException: com.mysql.jdbc.Driver2
at java.net.URLClassLoader.findClass(URLClassLoader.java:381)
at java.lang.ClassLoader.loadClass(ClassLoader.java:424)
at sun.misc.Launcher$AppClassLoader.loadClass(Launcher.java:335)
at java.lang.ClassLoader.loadClass(ClassLoader.java:357)
at java.lang.Class.forName0(Native Method)

```

```
at java.lang.Class.forName(Class.java:264)
at com.gpcoder.SQLExceptionExample.main(SQLExceptionExample.java:18)
```

-Nếu password không đúng, ta sẽ nhận được log lỗi như sau:

```
java.sql.SQLException: Access denied for user 'root'@'localhost' (using password: YES)
at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:129)
at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:97)
at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:122)
at com.mysql.cj.jdbc.ConnectionImpl.createNewIO(ConnectionImpl.java:827)
at com.mysql.cj.jdbc.ConnectionImpl.(ConnectionImpl.java:447)
at com.mysql.cj.jdbc.ConnectionImpl.getInstance(ConnectionImpl.java:237)
at com.mysql.cj.jdbc.NonRegisteringDriver.connect(NonRegisteringDriver.java:199)
at java.sql.DriverManager.getConnection(DriverManager.java:664)
at java.sql.DriverManager.getConnection(DriverManager.java:247)
at com.gpcoder.SQLExceptionExample.main(SQLExceptionExample.java:20)
-Nếu sai câu lệnh SQL ta sẽ nhận được log lỗi tương tự sau:
java.sql.SQLSyntaxErrorException: Table 'jdbcdemo.user2' doesn't exist
at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:120)
at com.mysql.cj.jdbc.exceptions.SQLError.createSQLException(SQLError.java:97)
at com.mysql.cj.jdbc.exceptions.SQLExceptionsMapping.translateException(SQLExceptionsMapping.java:122)
at com.mysql.cj.jdbc.StatementImpl.executeUpdateInternal(StatementImpl.java:1335)
at com.mysql.cj.jdbc.StatementImpl.executeLargeUpdate(StatementImpl.java:2108)
at com.mysql.cj.jdbc.StatementImpl.executeUpdate(StatementImpl.java:1245)
at com.gpcoder.SQLExceptionExample.main(SQLExceptionExample.java:26)
```

Câu hỏi ôn tập

8. Giao dịch (Transaction) là gì?. Định nghĩa giao dịch trong ngữ cảnh của lập trình JDBC.
9. Tại sao cần sử dụng giao dịch trong JDBC?. Nêu rõ lợi ích và tầm quan trọng của việc sử dụng giao dịch trong lập trình JDBC.
10. Các thuộc tính ACID là gì?. Mô tả ý nghĩa của mỗi thuộc tính ACID (Atomicity, Consistency, Isolation, Durability) khi áp dụng cho giao dịch.
11. Làm thế nào để bắt đầu một giao dịch trong JDBC?. Mô tả cách bắt đầu một giao dịch bằng JDBC và cách kiểm soát quá trình này.
12. Cách commit (hoàn thành) một giao dịch trong JDBC?. Mô tả quá trình commit giao dịch và tại sao nó quan trọng.

13. Nêu rõ khái niệm Rollback trong giao dịch JDBC ?. Giải thích ý nghĩa và cách sử dụng Rollback trong JDBC.

BÀI TẬP THỰC HÀNH

Bài 1.

Tạo một cơ sở dữ liệu SinhVienDataBase và tạo bảng "students" với các trường như "id", "name", "age", và "grade" trên các hệ quản trị cơ sở dữ liệu khác nhau như MySQL, SQL server.

-Xây dựng các phương thức thực hiện khởi tạo đối tượng Connection theo như mẫu sau:

```
-public static Connection getMySQLConnection(String hostName, String dbName,
String userName, String password) throws SQLException,
ClassNotFoundException {...}
```

-Xây dựng phương thức sử dụng đối tượng **ResultSet** hiển thị thông tin của bảng ra *students*.

```
public void showData(ResultSet rs) throws SQLException{...}
```

Bài 2.

1. Xây dựng các phương thức thực hiện kết nối với cơ sở dữ liệu và thực hiện các phương thức thao tác (thêm mới, cập nhập, sửa, xóa...) với bản ghi trong bảng cơ sở dữ liệu.

```
public interface DAO<T> {
    public ArrayList<T> getAll(Connection con);
    public void insert(T ob,Connection con);
    public void updateC(T ob,Connection con) ;
    public boolean Delete(T ob,Connection con);
    public Object find(int id,Connection con);
    public Object findByName(String name,Connection con);
}
```

Bài 3.

Tổng hợp các kiểu ngoại lệ Exception và SQLException trong kết nối và truy vấn cơ sở dữ liệu.

Bài 4.

Xây dựng một ứng dụng đơn giản để quản lý giao dịch ngân hàng bằng Java, sử dụng JDBC và giao dịch để đảm bảo tính nhất quán trong cơ sở dữ liệu tương ứng với cơ sở dữ liệu như sau:

```
CREATE DATABASE banking;
USE banking;
CREATE TABLE accounts (
```

```
account_number INT PRIMARY KEY,  
customer_name VARCHAR(255),  
balance DECIMAL(10, 2)  
);  
INSERT INTO accounts VALUES (1, 'John Doe', 1000.00);  
INSERT INTO accounts VALUES (2, 'Jane Smith', 2000.00);
```

-Xây dựng phương thức để chuyển tiền từ tài khoản 1 sang tài khoản 2 sử dụng các khái niệm của Transaction và phương thức hiển thị số dư tài khoản.

